Shred Vault Security Report

Conducted by:
David Lumala
Senior Security Researcher
Telegram: @DLumala
LinkedIn: Eng.David Lumala

Jan 2026

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. We ultimately can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts/code . Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended. Any slight change of code and versions automatically requires a new thorough review. A total of **11** issues were uncovered.

About Shred Vault

Shred is a yield-bearing USDC vault that mints shUSD, an upgradeable ERC-20 token where deposits accrue interest via a liquidity index and can be made with optional referrals and EIP-2612 permits. It features flexible withdrawals with price-impact fees and immediate-or-queued fulfillment, operator-managed liquidity and sweep-to-strategy, configurable caps and minimums.

Security Assessment Summary

Review commit hash:
• 4532751932739d24868de333a69cc8b68316072f
(redshift-labs/shred_contracts)

Fixes review commit hash:
• 2f80dc75bb3f38a17db96c2cb5149d8803f1c1d8
(redshift-labs/shred_contracts)

Scope

- `ShredVault.sol`
- `ICreateX.sol`
- `MathUtils.sol`

Findings

[L-01] Price impact amount rounds in favor of user

Resolved

In `requestWithdrawal()`, the calculation of the price impact amount rounds down.

```
        uint256 priceImpactAmount = (grossAssets * $.priceImpact) / BPS_DIVISOR;
```

Given that there is no minimum withdrawal amount, this means that users can technically withdraw amounts without having any price impact deducted. For example, if the price impact is 2%, a user withdrawing 0.000049 USDC would have a price impact amount of zero (rounded down), and thus receive the full amount without any deduction.

While gas fees wouldn't make this strategy economically viable, it is recommended to address this by rounding up to ensure that every calculation rounds in favor of the protocol.

[L-02] APY discrepancy based on interaction frequency

Acknowledged

The protocol uses a Taylor series approximation for per-second interest compounding. The documentation states that this implementation offers "accurate yield regardless of interaction frequency".

However, the formula offers varying APY based on the frequency of index updates. For instance, with a 13.98% APR:

- Updating the index every block over a year results in an APY of approximately 15.0044%.
- Updating the index only once after a year results in an APY of approximately 15.0027%.

While the yield difference is negligible, it contradicts the claim of "accurate yield regardless of interaction frequency". It is important to be aware of this discrepancy, especially if off-chain services rely on accurate yield calculations.

Proof of Concept

```
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import {MathUtils} from "../src/libraries/MathUtils.sol";

contract PoC is Test {
    function test_diff_getCurrentIndex() public pure {
        uint256 rate = 0.1398e27; // 13.98% APR
        uint256 elapsed = 365 days;

        // A) multiple updates
        uint256 liquidityIndexA = 1e27;
        uint256 blockRate = 12 seconds;
```

```
        for (uint256 i = 0; i < elapsed; i += blockRate) {
            uint256 compoundFactorA = MathUtils.calculateCompoundedInterest(rate, 0,
blockRate);
            liquidityIndexA = MathUtils.rayMul(liquidityIndexA, compoundFactorA);
        }
        console.log("Index A:", liquidityIndexA);

        // B) single update
        uint256 liquidityIndexB = 1e27;
        uint256 compoundFactorB = MathUtils.calculateCompoundedInterest(rate, 0, elapsed);
        liquidityIndexB = MathUtils.rayMul(liquidityIndexB, compoundFactorB);
        console.log("Index B:", liquidityIndexB);

        assert(liquidityIndexA > liquidityIndexB);
    }
}
```

Shred comments

Status: Acknowledged (Will Not Fix)

Response:
We acknowledge this finding and consider it expected behavior.
The variance observed between update frequencies is within our documented "~99.9999% accuracy" bound for the Taylor series approximation.

[L-03] Unclaimed USDC from blocked/blacklisted users cannot be recovered

Acknowledged

`ShredVault.fulfillWithdrawal()` and `ShredVault.fulfillWithdrawalBatch()` allows fulfilling to the user with pending requests and not allowing the users to claim USDC if they got blocked.

This issue applies to three scenarios:

1. User was already blocked when operator fulfilled the withdrawal
2. User is blocked **after** operator fulfills but **before** user claims
3. User becomes USDC blacklisted

This can cause the unclaimed `USDC` to be permanently trapped in the vault and counted in `totalClaimableWithdrawals`, reducing available liquidity for `sweepToStrategy()` operations.

ShredVault.sol#L431-L454 (also see ShredVault.sol#L461-L513)

```
function fulfillWithdrawal(address user, uint256 assets) external onlyRole(OPERATOR_ROLE)
nonReentrant {
    ShredVaultStorage storage $ = _getShredVaultStorage();

    //--- SNIPPED ---

@>  $.claimableWithdrawals[user] += assets;
@>  $.totalClaimableWithdrawals += assets;

    emit WithdrawalFulfilled(user, assets);
}
```

When a blocked user attempts to claim, the transaction reverts:

ShredVault.sol#L409-L423

```
function claimWithdrawal() external whenNotPaused nonReentrant {
    ShredVaultStorage storage $ = _getShredVaultStorage();

    // Blocklist check - blocked users cannot claim (security measure)
@>  _revertIfBlocked(msg.sender); //@audit reverts for blocked users

    //--- SNIPPED ---
@>  $.usdc.safeTransfer(msg.sender, claimable); //@audit reverts for blacklisted users
    emit WithdrawalClaimed(msg.sender, claimable);
}
```

Since `totalClaimableWithdrawals` includes funds for blocked/blacklisted users who can claim, the actual sweepable amount is count not reduced through `ShredVault.sweepToStrategy()`.

Recommendations

There are muiltiple approaches to adopt as per design decision:

1. Consider adding a rescue function to rescue stuck `USDC` from blocked/blacklisted users.
2. Consider adding blocklist and or blacklisting validation in both fulfillment functions before updating claimable state. This prevents blocked users from entering the claimable state entirely but if there are the cases where the users are blocked/blacklisted after the fulfilment, the rescue function will be the second step to rescue the stuck `USDC`.

Shred comments

Status: Acknowledged (Will Not Fix)

Response:
This is intentional design for compliance reasons. Blocked users' funds remain quarantined in the contract rather than being released or rescued back into protocol liquidity. The operator retains discretion to not fulfill pending withdrawals for already-blocked users. The reduced sweep capacity is an acceptable trade-off for regulatory safety.

[L-04] Dust transfer enables suppression of referral event emission in `ShredVault._deposit()`

Resolved

`ShredVault._deposit()` uses `balanceOf(receiver) == 0` to determine whether a user is new and should trigger a referral event. Since `shUSD` is a standard ERC20 token with unrestricted transfer functionality, anyone can send a minimal amount (1 wei) of `shUSD` to a target address before their first deposit, causing the `isNewUser` check to fail and suppressing the `Referral` event emission.

The referral event is intended to track new user acquisition (or re-referral after a user fully exits) for off-chain reward distribution to referrers.

[ShredVault.sol#L253-L284]

```
function _deposit(uint256 assets, address receiver, address referrer) internal returns
(uint256 shares) {
    //--- SNIPPED ---

    _updateIndex();

    // Check deposit cap against total value of all shUSD (includes accrued yield)
    if (totalValue() + assets > $.maxTotalDeposits) revert MaxDepositsExceeded();

    // Check if receiver is a new user before minting
    bool isNewUser = balanceOf(receiver) == 0;

    //--- SNIPPED ---

    // Emit referral for new users with valid non-self referrer
    if (referrer != address(0) && referrer != receiver && isNewUser) {
        emit Referral(receiver, referrer, assets);
    }
}
```

Consider applying an explicit mapping to track whether a user is a new user, independent of their token balance.

For example, add a `mapping(address => bool) isNewUser` state variable and set it to

- `true` when a user first deposits.
- `false` to reset once the user makes a withdrawal that makes their balance drop to zero (fully exit).

[L-05] Stale liquidity index in view functions

Resolved

`liquidityIndex()` and `getYieldState()` return the stored liquidity index `$.liquidityIndex` instead of the current index that includes interest accrued up to the current block.

The contract already exposes the correct, up to date index internally via `_getCurrentIndex()`, which is used by `convertToAssets()`, `convertToShares()`, and `totalValue()`. External callers that use `liquidityIndex()` or `getYieldState().index` for display or calculations will see a value that does not match the exchange rate implied by `convertToAssets()` / `convertToShares()`, leading to wrong UX or off-chain logic. The NatSpec for `getYieldState()` describes the returned index as `Current liquidity index`, which reinforces the expectation of a live index.

Recommendation: Update `liquidityIndex()` and `getYieldState()` to return the result of `_getCurrentIndex()`, or explicitly document that the returned index is the stored value and expose a new view function `getCurrentIndex()` that returns the correct up to date value.

[L-06] `requestWithdrawal()` has no protection for instant vs pending outcome

Resolved

Between calling `previewWithdrawal()` and `getAvailableWithdrawalCapacity()` and executing `requestWithdrawal()`, vault liquidity can change (e.g. another user withdraws). The caller may expect immediate withdrawal based on the view results but end up with a pending withdrawal instead. There is no parameter to protect the user in case of such an unexpected outcome.

Example: A user checks `getAvailableWithdrawalCapacity()` and sees enough USDC to cover their needed liquid assets. Before their `requestWithdrawal(shares)` is executed, another withdrawal consumes the liquidity. The user's request is then queued as pending. They might have preferred to not burn shares as they could get liquid assets from another protocol.

Recommendation: Consider adding a flag such as `requireImmediate` that reverts when set to `true` and the withdrawal is set as pending. This lets users align execution with their expectation and avoid undesired pending outcomes.

[L-07] `requestWithdrawal()` does not allow partial fulfillment

Acknowledged

`requestWithdrawal()` is all or nothing, if the vault has some but not enough USDC to cover the full requested amount, the user receives nothing immediately and the entire `netAssets` amount is queued as pending. The user must wait for the full amount instead of receiving the available portion now and only waiting for the illiquid remainder.

A partial fullfillment can be currently achieved by splitting into multiple `requestWithdrawal()` smaller calls, but that is not the best UX.

Consider allowing partial fulfillment. When `availableUsdc > 0` but `availableUsdc < netAssets`, send `availableUsdc` to the user immediately and only add `(netAssets - availableUsdc)` to `pendingWithdrawals`. This aligns with the design goal of "instant when possible".

Shred comments

Status: Acknowledged (Will Not Fix)

Response:

This behavior is intentional. The all-or-nothing approach:

- Protects smaller withdrawers — If a large withdrawal could drain all available liquidity via partial fulfillment, subsequent smaller users would be forced into the async queue. The current design preserves instant withdrawal capacity for users whose requests fit within available balance.
- Prevents whale-driven capacity exhaustion — A single large withdrawal shouldn't deplete liquidity for the majority of users.
- Users retain flexibility — Those who want partial fulfillment can split their request into multiple smaller requestWithdrawal() calls.

The "instant when possible" design goal applies to requests that can be fully serviced, not partial claims on shared liquidity.

[L-08] No slippage protection against `priceImpact` change in withdrawals

Resolved

Description

The `requestWithdrawal()` function applies the `priceImpact` fee when calculating withdrawal amounts, but lacks slippage protection.
Admin can change the `priceImpact` parameter between user's transaction submission and execution, causing users to receive less USDC than expected.

Recommendation

Add `minNetAssets` parameter for slippage protection in `requestWithdrawal` function.

[L-09] Missing public index update function

Acknowledged

Description

The `_updateIndex()` function has overflow protection that reverts if the new index exceeds 10x the old index:

```
function _updateIndex() internal {
    // ...
    // Overflow protection: index should not grow more than 10x in a single update
    if (newIndex > $.liquidityIndex * 10) {
        revert InterestCalculationOverflow();
    }
    ...
}
```

The Problem:

- `_updateIndex()` is only called during deposits and withdrawals
- No standalone public function exists to update the index without a deposit/withdrawal
- If the protocol has no activity for an extended period, accumulated interest could exceed 10x growth

Recommendation

Add a public update function so that anyone (users, bots, admin) can proactively update the index during low-activity periods without any deposit or withdraw:

```
/// @notice Update the liquidity index to reflect accrued interest
/// @dev Anyone can call this to prevent index overflow during periods of inactivity
function updateIndex() external nonReentrant {
    _updateIndex();
}
```

Shred comments

Status: Acknowledged (Will Not Fix)

Response:
Unrealistic scenario. At maximum rate (100% APR), complete protocol inactivity for ~2.3 years would be required to trigger the overflow protection. At planned rates (~14% APR), this extends to 16+ years.
Any deposit or withdrawal activity resets the accumulation window. A protocol with zero activity for multiple years has larger concerns than index overflow. Adding a public function increases surface area without meaningful benefit.

[L-10] Pause modifier inconsistency in withdrawal

Resolved

Description

The withdrawal fulfillment and claiming functions have inconsistent pause modifier usage:

- `fulfillWithdrawal()` - **NO** `whenNotPaused` modifier
- `fulfillWithdrawalBatch()` - **NO** `whenNotPaused` modifier
- `claimWithdrawal()` - **HAS** `whenNotPaused` modifier

This creates a scenario where when the contract is paused:

1. Operators can still fulfill pending withdrawals (moving USDC from pending → claimable)
2. Users **cannot** claim their fulfilled withdrawals
3. User funds become trapped in claimable state until contract is `unpaused`

Recommendations

- Remove the `whenNotPaused` modifier from `claimWithdrawal()`.
- Add `whenNotPaused` modifier to both fulfill functions.

[L-11] New withdrawal requests can take precedence over pending withdrawals

Acknowledged

`requestWithdrawal()` completes the withdrawal immediately if there are sufficient funds in the vault and queues it otherwise.

The problem is that for the calculation of available USDC, only `totalClaimableWithdrawals` is considered, and not `totalPendingWithdrawals`.

```
    uint256 availableUsdc = vaultBalance > $.totalClaimableWithdrawals
        ? vaultBalance - $.totalClaimableWithdrawals
        : 0;
```

This means that when USDC is added to the contract balance by deposits or by funds returned from the strategy, new withdrawal requests may be fulfilled ahead of pending withdrawals.

Consider the following scenario:

1. Alice requests a withdrawal of 100 USDC, which is queued as pending because there are insufficient funds in the vault.
2. After one day, the balance of the vault increases by 100 USDC due to a deposit or the operator transferring funds back from the strategy.
3. Before the operator fulfills Alice's withdrawal, Bob requests a withdrawal of 100 USDC, which is completed immediately.
4. Alice has to keep waiting until new funds are available.

Proof of Concept

```
function test_frontRunFulfillment() public {
    // User1 and User2 deposit 1,000 USDC each
    uint256 depositAmount = 1000e6;
    vm.startPrank(user1);
    usdc.approve(address(vault), depositAmount);
    uint256 sharesUser1 = vault.deposit(depositAmount, user1);
    vm.stopPrank();

    vm.startPrank(user2);
    usdc.approve(address(vault), depositAmount);
    uint256 sharesUser2 = vault.deposit(depositAmount, user2);
    vm.stopPrank();

    // USDC is moved to strategy
    vm.prank(operator);
    vault.sweepToStrategy(depositAmount * 2);
```

```
    // User1 requests withdrawal (pending)
    vm.prank(user1);
    (uint256 netAssets, bool claimed) = vault.requestWithdrawal(sharesUser1);
    assertFalse(claimed);

    // Operator sends funds back to vault to fulfill withdrawals
    // (or a new deposit from another user is made)
    usdc.mint(address(vault), netAssets);

    // User2 withdraws instantly before User1's pending withdrawal is fulfilled
    vm.prank(user2);
    (, claimed) = vault.requestWithdrawal(sharesUser2);
    assertTrue(claimed);

    // User1 withdrawal cannot be fulfilled
    vm.prank(operator);
    vm.expectRevert();
    vault.fulfillWithdrawal(user1, netAssets);
}
```

Recommendations

```
    function requestWithdrawal(
(...)

-       uint256 availableUsdc = vaultBalance > $.totalClaimableWithdrawals
+       uint256 availableUsdc = vaultBalance > $.totalClaimableWithdrawals +
$.totalPendingWithdrawals

-           ? vaultBalance - $.totalClaimableWithdrawals
+           ? vaultBalance - $.totalClaimableWithdrawals - $.totalPendingWithdrawals
            : 0;
(...)
    function getAvailableWithdrawalCapacity() external view returns (uint256) {
        ShredVaultStorage storage $ = _getShredVaultStorage();
        uint256 balance = $.usdc.balanceOf(address(this));

-       return balance > $.totalClaimableWithdrawals ? balance - $.totalClaimableWithdrawals
: 0;
+       return balance > $.totalClaimableWithdrawals + $.totalPendingWithdrawals
+           ? balance - $.totalClaimableWithdrawals - $.totalPendingWithdrawals
+           : 0;
```

Shred comments

Status: Acknowledged (Will Not Fix)

Updated Docs for clarity - commit: https://github.com/redshift-labs/shred_contracts/commit/f9ab1c27bd6226493ebe532ef8949bf8a637f208

Response:

We acknowledge this finding. This is an intentional design decision with the following rationale:

1. UX Optimization for the Majority
   The protocol targets instant withdrawals for ~99% of users. By allowing new withdrawals to complete instantly from available USDC (including recent deposits), we optimize UX for the majority rather than queuing everyone behind pending users.
2. Pending Users Have Reduced Time Sensitivity
   When a withdrawal goes to pending, the UI displays an estimated fulfillment window (up to 24 hours). These users are not likely actively monitoring - e.g they'll check back once before and once after the estimation. The marginal "early fulfillment" benefit is outweighed by instant UX for newly withdrawing users.
3. Atomic Operator Fulfillment Prevents Race Conditions
   The vault operator can execute USDC return + batch fulfillment in a single atomic multicall transaction. The "operator sends funds, then someone front-runs" scenario described in the PoC doesn't occur in practice with current implementation - funds arrive and are immediately allocated to pending users atomically.
4. Deposit Netting is Also Intentional
   We also don't auto-fulfill pending withdrawals on new deposit in the SC code for the same reason - optimizing instant UX by allowing new deposits and withdrawals to net out.

Conclusion
Updated the docs to give more details on this design trade-off. The architecture optimizes for instant UX for the majority while pending users are serviced within the stated on UI 24-hour estimate through atomic operator fulfillment.